INGENIERÍA EN SOFTWARE

UML y Modelado

Bloque 01 · Apunte completo de estudio

Diagramas de clases · Diagramas de secuencia · Casos de uso
Diagramas de estado · Diagramas de actividad

Instituto Técnico Superior Córdoba

Ingeniería en Software · Prof. Mateo Alexis Vranich Prieto · Córdoba 2026

#	Sección
01	UML y modelado de software
02	Diagrama de clases y relaciones
03	Diagrama de casos de uso
04	Diagrama de secuencia
05	Diagrama de estados
06	Diagrama de actividad

01 · UML y modelado de software

Este bloque cubre el Lenguaje de Modelado Unificado (UML): la herramienta estándar para diseñar y comunicar la estructura y el comportamiento de un sistema antes de escribir una sola línea de código. Todos los ejemplos usan escenarios cotidianos: lomiterías, choripanes, fernetcolas y la vida cordobesa.

¿Qué es UML?

UML (Unified Modeling Language) es el lenguaje estándar para visualizar, especificar y documentar el diseño de sistemas de software. No es un lenguaje de programación: es un lenguaje de **diagramas**. Así como un arquitecto dibuja planos antes de construir una casa, un ingeniero de software dibuja diagramas UML antes de escribir código.

Pensalo así: si vas a abrir una lomitería en Nueva Córdoba, no empezás poniendo ladrillos. Primero planificás cuántas mesas van, dónde va la parrilla, cómo circula la gente, cuántos empleados necesitás. UML es ese plano, pero para software.

¿Para qué sirve modelar?

Modelar antes de codificar permite detectar errores de diseño cuando son baratos de corregir (un diagrama se borra y se rehace en minutos; refactorizar un sistema en producción puede llevar semanas). Los principales beneficios son:

- **Comunicar decisiones de diseño** sin ambigüedad entre desarrolladores, clientes y stakeholders.
- **Detectar problemas temprano:** dependencias circulares, responsabilidades mal asignadas, flujos incompletos.
- **Documentar la arquitectura** para que nuevos miembros del equipo entiendan el sistema rápidamente.
- **Validar con el cliente:** un diagrama de casos de uso es mucho más fácil de leer que un SRS de 80 páginas.

Tipos de diagramas UML

UML define 14 tipos de diagramas, pero en la práctica se usan regularmente unos pocos. Se dividen en dos grandes categorías:

Categoría	Diagrama	¿Qué muestra?	Analogía lomitería
Estructura (qué ES)	Clases	Clases, atributos, métodos y relaciones	El organigrama: quién es quién
Estructura (qué ES)	Casos de uso	Qué puede hacer cada actor	Qué hace el cliente, el cocinero, el cajero

Categoría	Diagrama	¿Qué muestra?	Analogía lomitería
Comportamiento (qué HACE)	Secuencia	Orden de mensajes entre objetos	Cliente pide, cocinero prepara, cajero cobra
Comportamiento (qué HACE)	Estado	Estados por los que pasa un objeto	Pedido: Recibido, En preparación, Listo, Entregado
Comportamiento (qué HACE)	Actividad	Flujo de trabajo completo	Proceso completo de preparar un lomito

02 · Diagrama de clases y relaciones

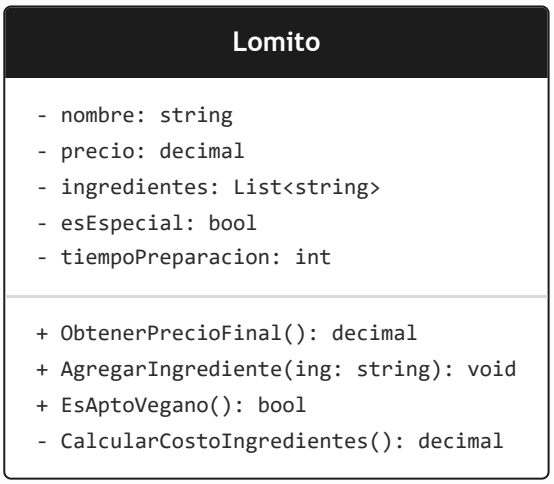
Es el diagrama más importante y más usado de UML. Muestra las clases del sistema, sus atributos, sus métodos y las relaciones entre ellas. Es la columna vertebral del diseño orientado a objetos.

Cada clase se representa como un rectángulo dividido en tres partes: el nombre arriba, los atributos en el medio y los métodos abajo. Los modificadores de acceso:

Símbolo	Significado	En C#	Ejemplo lomitería
+	Público: accesible desde cualquier lado	`public`	El menú: todos lo ven
−	Privado: solo dentro de la clase	`private`	La receta secreta del lomito
#	Protegido: la clase y sus hijas	`protected`	La receta que le pasa al hijo que hereda el negocio

Ejemplo: clase Lomito

Imaginemos que estamos diseñando el sistema de gestión para "La Lomi de la Cañada", una lomitería cordobesa. La clase `Lomito` en UML:



Las tres secciones de una clase: nombre, atributos y métodos.

En código C#, esa clase se traduce directamente:

C#

```
public class Lomito {
    private string _nombre;
    private decimal _precio;
    private List<string> _ingredientes;
    private bool _esEspecial;
    private int _tiempoPreparacion;

    public decimal ObtenerPrecioFinal() { ... }
    public void AgregarIngrediente(string ing) { ... }
    public bool EsAptoVegano() { ... }
    private decimal CalcularCostoIngredientes() { ... }
}
```

Relaciones entre clases

Las clases no existen aisladas: se relacionan entre sí. En UML hay varios tipos de relaciones, cada una con un significado distinto y una notación específica.

Relación	Significado	Flecha UML	Ejemplo cordobés
Asociación	"Conoce a" / "usa a". Relación genérica.	Línea simple con flecha	Un Cliente hace un Pedido
Agregación	"Tiene" (débil). El contenido existe sin el contenedor.	Rombo vacío	Una Lomitería tiene Empleados. Si cierra, los empleados siguen existiendo.
Composición	"Tiene" (fuerte). El contenido NO existe sin el contenedor.	Rombo lleno	Un Pedido tiene Líneas de pedido. Si se borra el pedido, las líneas desaparecen.
Herencia	"Es un tipo de". La subclase hereda de la superclase.	Flecha triangular	Un LomitoEspecial es un tipo de Lomito. Un LomitoÁrabe también.
Implementación	"Cumple el contrato de". Clase implementa interfaz.	Flecha triangular punteada	PagoEfectivo y PagoMercadoPago implementan IMetodoPago
Dependencia	"Usa temporalmente". Relación débil y transitoria.	Flecha punteada	El ServicioDelivery usa un CalculadorDistancia pero no lo guarda.

Multiplicidad (cardinalidad)

La multiplicidad indica cuántos objetos participan en cada extremo de una relación:

Notación	Significado	Ejemplo
1	Exactamente uno	Un pedido tiene 1 cliente

Notación	Significado	Ejemplo
0..1	Cero o uno (opcional)	Un lomito tiene 0..1 salsa extra
*****	Cero o muchos	Un cliente tiene * pedidos
1..*	Uno o muchos (al menos uno)	Un pedido tiene 1..* líneas (al menos un ítem)

> **Regla práctica para distinguir agregación de composición:** preguntate "¿si destruyo el contenedor, el contenido sigue existiendo?". Si la lomitería cierra, los empleados buscan otro trabajo (agregación). Si se borra un pedido, sus líneas de detalle no tienen sentido solas (composición).

03 · Diagrama de casos de uso

Muestra **qué puede hacer cada actor** con el sistema, sin entrar en cómo lo hace. Es el diagrama más útil para comunicarse con el cliente porque es simple y no requiere conocimientos técnicos.

Elementos

- **Actor:** quien interactúa con el sistema (persona, otro sistema). Se dibuja como un muñeco de palitos.
- **Caso de uso:** una funcionalidad del sistema. Se dibuja como una elipse con el nombre adentro.
- **Sistema:** rectángulo que delimita los límites del sistema.
- **Relaciones:** líneas que conectan actores con casos de uso, o casos de uso entre sí (include, extend).

Include vs Extend

Relación	Significado	Ejemplo lomitería
<<include>>	El caso base **siempre** incluye al otro. Es obligatorio.	"Hacer pedido" include "Verificar stock". No podés hacer un pedido sin verificar ingredientes.
<<extend>>	El caso base **puede** incluir al otro. Es opcional.	"Hacer pedido" extend "Aplicar cupón de descuento". El cupón es opcional.

Ejemplo: sistema de la lomitería

Los actores: **Cliente** (pide, paga), **Cocinero** (prepara, gestiona stock) y **Administrador** (gestiona menú, ve reportes).

Actor	Casos de uso
Cliente	Ver menú · Hacer pedido · Elegir método de pago · Cancelar pedido · Dejar reseña
Cocinero	Ver pedidos pendientes · Marcar pedido como listo · Reportar falta de ingrediente
Administrador	Agregar/editar ítems del menú · Definir precios · Ver reportes de ventas · Gestionar empleados

> **Tip para el parcial:** un error común es confundir actores con usuarios. Un actor es un **rol**, no una persona. La misma persona puede ser Cliente al mediodía cuando pide un lomito, y Administrador a la noche cuando cierra la caja.

04 · Diagrama de secuencia

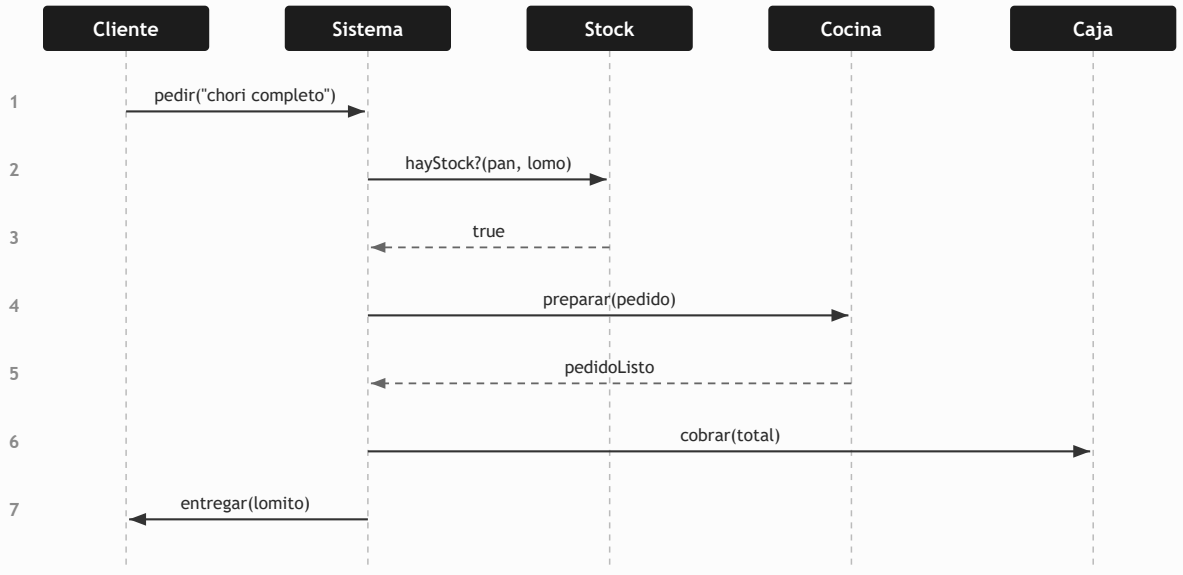
Muestra **cómo interactúan los objetos en el tiempo** para cumplir un caso de uso específico. El eje vertical es el tiempo (de arriba hacia abajo) y el eje horizontal muestra los participantes.

Elementos principales

- **Participante (lifeline):** un objeto o actor. Rectángulo arriba con una línea punteada vertical (su "línea de vida").
- **Mensaje sincrónico:** flecha con punta llena. El emisor espera la respuesta.
- **Retorno:** flecha punteada. La respuesta a un mensaje.
- **Fragmento alt:** como un if/else. Muestra caminos alternativos.
- **Fragmento loop:** indica repetición de una secuencia.

Ejemplo: flujo de un pedido de choripán

Un cliente pide un choripán completo en La Lomi de la Cañada. Los números a la izquierda indican el orden temporal:



Leé de arriba hacia abajo: (1) el cliente pide, (2) el sistema verifica stock, (3) stock confirma, (4) se envía a cocina, (5) cocina avisa que está listo, (6) se cobra en caja, (7) se entrega al cliente.

> **Tip:** Si no hubiera stock, el diagrama tendría un fragmento **alt** mostrando un camino alternativo donde se informa al cliente y no se continúa.

05 · Diagrama de estados

Muestra los **estados por los que pasa un objeto** a lo largo de su vida y las **transiciones** (eventos) que provocan el cambio de un estado a otro.

Elementos

- **Estado:** rectángulo redondeado con el nombre del estado.
- **Transición:** flecha etiquetada con el evento que provoca el cambio.
- **Estado inicial:** círculo negro sólido. Indica dónde empieza.
- **Estado final:** círculo negro dentro de un aro. Indica dónde termina.
- **Guardas:** condiciones entre corchetes [condición] que deben cumplirse.

Ejemplo: estados de un pedido en la lomitería

Un pedido en "La Lomi de la Cañada" pasa por estos estados:

Estado	Transición (evento)	Siguiente estado
(inicio)	Cliente hace el pedido	**Recibido**
Recibido	confirmar()	**En preparación**
En preparación	terminar()	**Listo para retirar**
Listo para retirar	entregar()	**Entregado** (fin)
Cualquier estado (excepto Entregado)	cancelar() [motivo]	**Cancelado** (fin)

Notar que "Cancelado" puede alcanzarse desde cualquier estado previo a "Entregado". La guarda **[motivo]** indica que se requiere una razón para cancelar.

> **¿Cuándo usar diagrama de estados?** Cuando un objeto tiene un ciclo de vida con estados claros y reglas sobre qué transiciones son válidas. Pedidos, tickets de soporte, cuentas de usuario, procesos de pago: todos son buenos candidatos.

06 · Diagrama de actividad

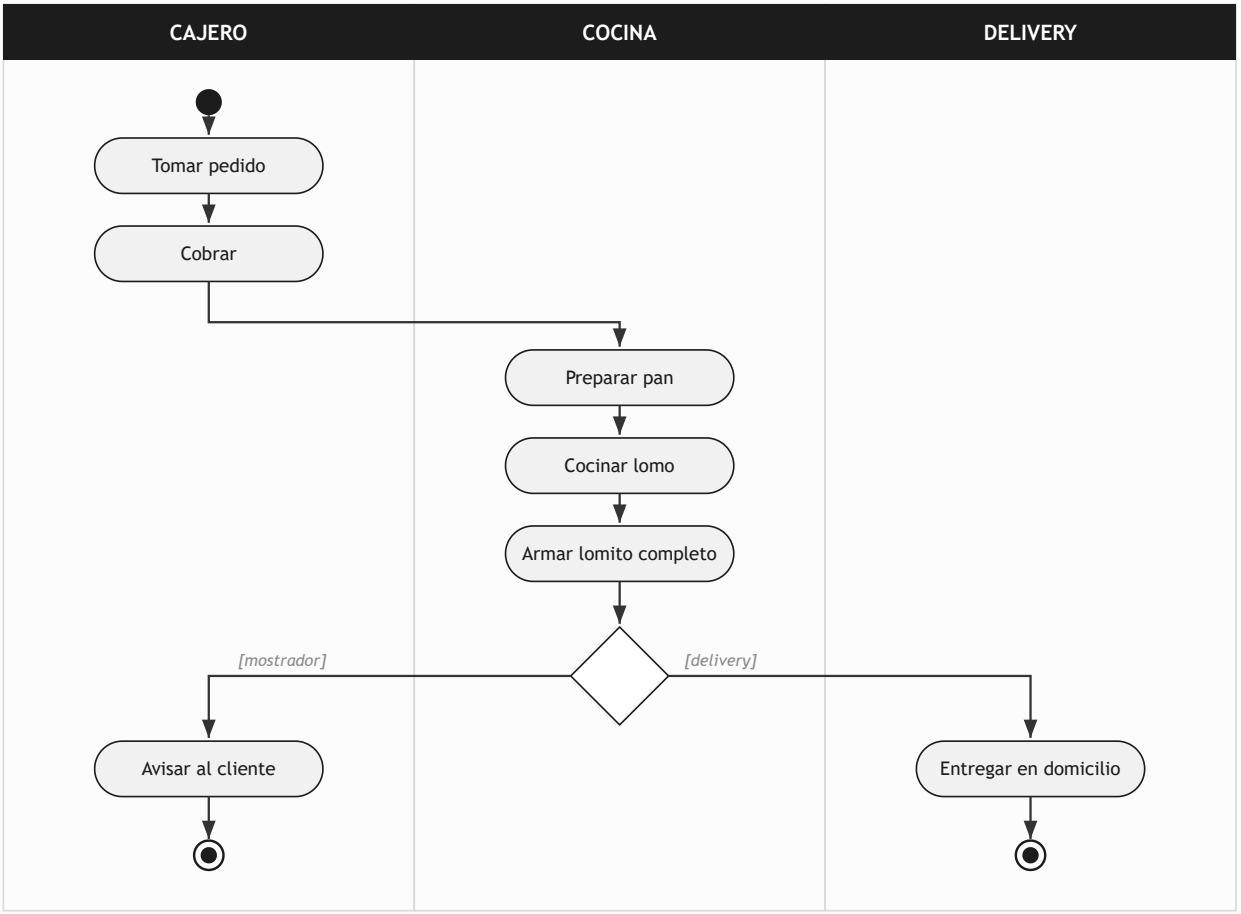
Es el más parecido a un **diagrama de flujo** clásico. Muestra el flujo de trabajo de un proceso, incluyendo decisiones y acciones paralelas. Usamos **swimlanes** (calles) para separar quién hace cada actividad.

Elementos

- **Actividad:** rectángulo redondeado. Una acción o tarea.
- **Decisión:** rombo. Bifurca el flujo según una condición.
- **Swimlanes:** columnas que indican quién hace cada actividad.
- **Nodo inicial / final:** círculo negro sólido (inicio) y círculo con aro (fin).
- **Fork/Join:** barra gruesa que indica actividades en paralelo.

Ejemplo: preparación de un lomito completo

Desde que entra el pedido hasta que sale el lomito. Cada columna indica quién es responsable de cada paso:



Los swimlanes dejan claro quién es responsable de cada paso. El rombo bifurca el flujo: si es delivery, pasa al repartidor; si es para retirar en mostrador, el cajero avisa al cliente.

07 · Resumen, errores comunes y glosario

¿Qué diagrama uso?

Necesito…	Uso…	Ejemplo
Ver estructura de clases y relaciones	**Diagrama de clases**	Diseñar las entidades de la lomitería
Mostrar qué puede hacer cada usuario	**Casos de uso**	Presentarle al dueño qué va a poder hacer cada rol
Detallar el paso a paso de una interacción	**Secuencia**	Modelar qué pasa cuando un cliente hace un pedido
Mostrar los estados de un objeto en el tiempo	**Estados**	Pedido: recibido, preparando, listo, entregado
Modelar un flujo de trabajo completo	**Actividad**	Proceso completo de preparar un lomito

Errores comunes en parciales

- **Confundir agregación con composición.** Preguntate: "¿si destruyo el contenedor, el contenido sigue existiendo?". Si sí: agregación. Si no: composición.
- **Poner operaciones del sistema como actores** en casos de uso. "Base de datos" no es un actor; es parte del sistema.
- **Mezclar niveles de abstracción** en un diagrama de clases. Si tenés Lomito y CalcularIVA en el mismo diagrama, algo anda mal.
- **Olvidar la multiplicidad** en las relaciones. Un diagrama sin multiplicidad es incompleto.
- **Hacer diagramas de secuencia sin un caso de uso concreto.** Cada diagrama de secuencia modela UN flujo específico.

Glosario del bloque

Término	Definición
UML	Unified Modeling Language. Lenguaje estándar de diagramas para modelar software.
Multiplicidad	Cantidad de instancias en una relación (1, 0..1, *, 1..*).
Actor	Entidad externa que interactúa con el sistema.
Lifeline	Línea vertical punteada: la vida de un participante en diagrama de secuencia.
Swimlane	División en diagrama de actividad que indica el responsable de cada acción.

Término	Definición
Transición	Cambio de estado provocado por un evento, opcionalmente condicionado por una guarda.
Include	Relación obligatoria entre casos de uso: el caso base siempre ejecuta al incluido.
Extend	Relación opcional entre casos de uso: se ejecuta solo si se cumple una condición.